

SCJP 1.6 (CX-310-065 , CX-310-066)

Subject: Mock Questions Set-5

Total Questions : 60

Prepared by : <http://www.javacertifications.net>

SCJP 6.0: Full Length Mock Exam Set-5(60 Questions)

Questions Question - 1

What will happen when you attempt to compile and run the following code?

```
int Output = 10;
boolean b1 = false;

if((b1 == true) && ((Output += 10) == 20))
{
    System.out.println("We are equal " + Output);
}

else
{
    System.out.println("Not equal! " + Output);
}
```

- 1.Compilation error, attempting to perform binary comparison on logical data type.
- 2.Compilation and output of "We are equal 10".
- 3.Compilation and output of "Not equal! 20".
- 4.Compilation and output of "Not equal! 10".

Explanation :

D is the correct answer.

(b1 == true) is not true so the statement ((Output += 10) == 20) will not execute. because if first statement is true then only 2nd statement will execute in case of and(&&) condition.

Question - 2

Which of the following statements are correct?

- 1) If multiple listeners are added to a component only events for the last listener added will be processed
- 2) If multiple listeners are added to a component the events will be processed for all but with no guarantee in the order
- 3) Adding multiple listeners to a component will cause a compile time error
- 4) You may remove as well add listeners to a component.

1.1 and 4

- 2.1,2 and 3
- 3.2 and 4
- 4.2 and 3

Explanation :

C is the correct answer.

If multiple listeners are added to a component the events will be processed for all but with no guarantee in the order

Question - 3

Which of the following modifiers can be applied to a class that is not a nested class?

- 1.public
- 2.protected
- 3.private
- 4.static

Explanation :

A is the correct answer.

only public, abstract and final is permitted in case of outer class.

Question - 4

```
interface IFace{}
class CFace implements IFace{}
class Base{}
public class ObRef extends Base{
    public static void main(String argv[]){
        ObRef ob = new ObRef();
        Base b = new Base();
        Object o1 = new Object();
        IFace o2 = new CFace();
    }
}
```

Given the above classes which of the following code will compile without error?

- 1)o1=o2;
- 2)b=ob;
- 3)ob=b;
- 4)o1=b;

- 1.1,2,4
- 2.2,3,4
- 3.1,3,4
- 4.3,4

Explanation :

A is the correct answer.

1)o1=o2; 2)b=ob; 4)o1=b; are true because you can A Superclass Variable Can Reference a Subclass Object.

Question - 5

```
class A extends Thread {  
    private int i;  
    public void run() {i = 1;}  
    public static void main(String[] args) {  
        A a = new A();  
        a.run();  
        System.out.print(a.i);  
    }  
}
```

How many threads are created in this Program?

- 1.1
- 2.2
- 3.3
- 4.0

Explanation :

A is the correct answer.

Main thread is a thread and calling run methods will not create thread.

Question - 6

What will happen when you attempt to compile and run the following code?

```
class Base  
{  
    int i = 99;  
  
    public void amethod()  
    {  
        System.out.println("Base.amethod()");  
    }  
    Base()  
    {  
        amethod();  
    }  
}  
  
public class Derived extends Base  
{  
    int i = -1;
```

```

public static void main(String argv[])
{
    Base b = new Derived();
    System.out.println(b.i);
    b.amethod();
}

public void amethod()
{
    System.out.println("Derived.amethod()");
}
}

```

- 1.Derived.amethod() -1 Derived.amethod()
- 2.Derived.amethod() 99
- 3.Derived.amethod() 99
- 4.Derived.amethod()

Explanation :

A is the correct answer.

amethod is overridden so constructor of class Base also called amethod of subclass .

Question - 7

Which data type is wider for the purpose of casting: float or long?

- 1.float
- 2.long
- 3.none of the above
- 4.none of the above

Explanation :

A is the correct answer.

float is wider than long, because the entire range of long fits within the range of float.

Question - 8

Which statement is correct ?

- 1.If super class has different constructor other than default then in the sub class you can use default constructor
- 2.If super class has different constructor other than default then in the sub class you can't use default constructor
- 3.There is no relation between superclass and subclass.
- 4.none of the above

Explanation :

B is the correct answer.

If super class has different constructor other than default then in the sub class you can't use default constructor.

Question - 9

```
import java.util.*;

class C
{
    final Vector v;
    C()
    {
        v=new Vector();
    }
    C(int i)
    {

    }

    public void someMethod()
    {
        System.out.println(v.isEmpty());
    }
}
```

- 1.compile time error
- 2.runtime exception
- 3.the code compiles and runs fine
- 4.none of the above

Explanation :

A is the correct answer.

The blank final field v may not have been initialized. If you do C c= new C(5) then final variable will be not initialized , So this is giving compile error.

Question - 10

? Each element must be unique
? Duplicate elements must not replace old elements.
? Elements are not key/value pairs.
? Accessing an element can be almost as fast as performing a similar operation on an array.

Which of these classes provide the specified features?

1. LinkedList
2. TreeMap
3. HashMap
4. HashSet

Explanation :

D is the correct answer.

All the mentioned features are related to HashSet. ? Each element must be unique ? Duplicate elements must not replace old elements. ? Elements are not key/value pairs. ? Accessing an element can be almost as fast as performing a similar operation on an array.

Question - 11

What is the output for the below code ?

```
class Test {
public static void main (String[] a1) {
System.out.println(a1.length());           //1
System.out.println(a1[0]);                 //2
System.out.println(a1);                     //3
}}
```

1. compile time error at line1
2. compile time error at line2
3. compile time error at line3
4. Runtime exception

Explanation :

A is the correct answer.

length is not a method. it's a variable

Question - 12

Given the code below, and making no other changes, which access modifiers

(public, protected or private) can legally be placed before myMethod() on line 3?

If line 3 is left as it is, which keywords can legally be placed before myMethod on line 8?

1. class A
2. {
3. void myMethod() {}
4. }
- 5.

```

6.  class B extends A
7.  {
8.      void myMethod() {}
9.  }

```

- 1.private or nothing(i.e. leaving it as it is) on line 3. Nothing(i.e. leaving it as it is) or protected or public on line 8.
- 2.public or protected on line 3. private or nothing(i.e. leaving it as it is) on line 8.
- 3.nothing(i.e. leaving it as it is) or protected or public on line 3. private or nothing(i.e. leaving it as it is) on line 8.
- 4.None of the above.

Explanation :

A is the correct answer.

Methods may not be overridden to be more private. For example, a protected method can't be overridden by a default one. But you can override a default method to make it protected or public. .

Question - 13

What will be output by the following line of code?

```
System.out.println(010|4);
```

- 1.12
- 2.10
- 3.11
- 4.14

Explanation :

A is the correct answer.

No explanation available

Question - 14

In the following pieces of code, A and D will compile without any error. True/False?

```

A: StringBuffer sb1 = "abcd";
B: Boolean b = new Boolean("abcd");
C: byte b = 255;
D: int x = 0x1234;
E: float f1 = 1.2;

```

- 1.true
- 2.false
- 3.none of these

4.no idea

Explanation :

B is the correct answer.

A: StringBuffer sb1 = "abcd"; cannot convert from String to StringBuffer . C: byte b = 255; Type mismatch: cannot convert from int to byte. E: float fl = 1.2; Type mismatch: cannot convert from double to float. Others compile without any error.

Question - 15

What is the output ?

```
class Test
{
    public static void main(String a[])
    {
        Boolean b = new Boolean("abcd");
        System.out.println(b.booleanValue());
    }
}
```

- 1.true
- 2.false
- 3.compile with error.
- 4.Runtime Exception

Explanation :

B is the correct answer.

output is false because b.booleanValue() return false if Boolean constructor value other than "true";

Question - 16

Which of the following classes will not allow unsynchronized read operations by multiple threads?

- 1.Vector
- 2.TreeMap
- 3.TreeSet
- 4.HashMap

Explanation :

A is the correct answer.

All methods in Vector are synchronized.

Question - 17

```
class base
{
base(int c)
{
    System.out.println("base");
}
}
class Super extends base
{
    Super()
    {
        System.out.println("super");
    }
public static void main(String [] a)
{
    base b1=new Super();
}
}
```

- 1.compile time error
- 2.runtime exception
- 3.the code compiles and runs fine
- 4.None of the above

Explanation :

A is the correct answer.

If super class has constructor other than no-arg constructor then subclass can't use no-arg constructor. Either define a no-arg constructor in base, or call super(n) as the first statement in your constructors in Super.

Question - 18

What will happen if you attempt to compile and run the following code?

```
public class Test {

    public static void main(String argv[]){
        Integer ten=new Integer(10);
        Long nine=new Long (9);
        System.out.println(ten + nine);
        int i=1;
        System.out.println(i + ten);
    }
}
```

- 1.19 followed by 20

- 2.19 followed by 11
- 3.Compile time error
- 4.10 followed by 1

Explanation :

B is the correct answer.

From jdk 1.5 onwards : Unboxing conversion converts objects of wrapper types to values of corresponding primitive types.

Question - 19

What will happen when you attempt to compile and run the following code with the command line "hello there"

```
public class Arg{
String[] MyArg;
public static void main(String argv[]){
    MyArg=argv;
}
public void amethod(){
    System.out.println(argv[1]);
}
}
```

- 1.Compile with error
- 2.Compile successfully and output of "hello"
- 3.Compile successfully and output of "there"
- 4.None of the above

Explanation :

A is the correct answer.

Cannot make a static reference to the non-static field MyArg.

Question - 20

What will happen when you attempt to compile and run the following code

```
class Base{
    public void Base(){
        System.out.println("Base");
    }
}
public class In extends Base{
    public static void main(String argv[]){
        In i=new In();
    }
}
```

- 1.Compile with error, Base is a keyword
- 2.Compile successfully and no output at runtime
- 3.Print Base
- 4.Runtime error Base has no valid constructor

Explanation :

B is the correct answer.

Because the method in Base called Base has a return type it is not a constructor and there for does not get called on creation of an instance of its child class In

Question - 21

```
class c1
{
public static void main(String a[])
{
    System.out.println(Double.NaN==Double.NaN) ;
    System.out.println(Double.NaN!=Double.NaN) ;
    System.out.println(Float.NaN==Double.NaN) ;
}
}
```

- 1.prints false true false
- 2.print true false true
- 3.prints true true true
- 4.prints false false false

Explanation :

A is the correct answer.

Comparison of a double with double.NaN (Not a Number) will always return false.

Question - 22

When a byte is added to a char, what is the type of the result?

- 1.byte
- 2.int
- 3.long
- 4.non of the above

Explanation :

B is the correct answer.

The result of all arithmetic performed with the binary operators (not the assignment operators) is an int, a long, a float, or a double. Here byte and char are promoted to int, so the result is an int.

Question - 23

Which of the following statements are true?

- 1) At the root of the collection hierarchy is a class called Collection
- 2) The collection interface contains a method called enumerator
- 3) The iterator method returns an instance of the Vector class
- 4) The Set interface is designed for unique elements

- 1.1
- 2.2
- 3.3
- 4.4

Explanation :

D is the correct answer.

No explanation available

Question - 24

What will happen when you attempt to compile and run this code?

```
private class Base{}
public class Vis{
    transient int iVal;
    public static void main(String elephant[]){
    }
}
```

- 1.Compile time error: Base cannot be private
- 2.Compile time error indicating that an integer cannot be transient
- 3.Compile time error transient not a data type
- 4.Compile time error malformed main method

Explanation :

A is the correct answer.

outer class can't be private , only public, abstract and final allowed.

Question - 25

```
class A extends Thread {
```

```

private int i;
public void run() {i = 1;}
public static void main(String[] args) {
A a = new A();
a.run();
System.out.print(a.i);
}}

```

1. Prints nothing
2. Prints: 1
3. Prints: 01
4. Compile-time error

Explanation :

B is the correct answer.

a.run() method was called instead of a.start(); so the full program runs as a single thread so a.run() is guaranteed to complete.

Question - 26

```

class Test
{
    static int s;
    public static void main(String a[]){
        Test obj=new Test();
        obj.m1();
        System.out.println(s);
    }
    void m1()
    {
        int x=1;
        m2(x);
        System.out.println(x+"");
    }
    void m2(int x){
        x=x*2;
        s=x;
    }
}

```

1. prints 1,2
2. prints 2,0
3. prints 2,2
4. compile time error

Explanation :

A is the correct answer.

Only objects and arrays are passed by reference. other are passed by value. s is a static variable which is global to the class.

Question - 27

What happens if a class defines a method with the same name and parameters as a method in its superclass?

- 1.The compiler automatically renames the subclass method.
- 2.The program runs since because overriding methods is allowed
- 3.The program throws a DuplicateMethod exception.
- 4.The compiler shows a DuplicateMethod error.

Explanation :

B is the correct answer.

No explanation available

Question - 28

Fill in all the gaps

```
public class Test {  
  
    public static void main(String[] args) {  
  
        expr = 1;  
        switch (expr) {  
            case 1:  
                System.out.println("ONE"); break;  
            case 2:  
                System.out.println("TWO"); break;  
            case 3:  
                System.out.println("THREE"); break;  
            default:  
                assert false;  
        }  
    }  
}
```

Use the following fragments zero or many times

String

Short

Boolean

Float

Question - 29

What is the output ?

```
public class Test {
```

```

    public static void main(String[] args) {

        Integer i = null;
        int j = i;
        System.out.println(j);
    }

}

```

- 1.0
- 2.null
- 3.Compile with error
- 4.java.lang.NullPointerException

Explanation :

D is the correct answer.

trying to unboxing null so throws NullPointerException

Question - 30

What is the output of the bellow code ?

```

public class Test {

    public static void main(String[] args) {

        doSomething(1);
        doSomething(2.0);

    }

    public static void doSomething(double num) {
        System.out.println("double : " + num);
    }

    public static void doSomething(Integer num) {
        System.out.println("Integer : " + num);
    }

}

```

- 1.double : 1.0 double : 2.0
- 2.Integer : 1 double : 2.0
- 3.Integer : 1 Integer : 2
- 4.None of the above

Explanation :

A is the correct answer.

The method resolution includes three passes: 1. Attempt to locate the correct method without any boxing, unboxing, or vararg invocations. 2. Attempt to locate the correct method with boxing, unboxing, and without any vararg invocations. 3. Attempt to locate the correct method with boxing, unboxing, or vararg invocations.

Question - 31

What is the output ?

```
public class Test {  
  
    public static void main(String[] args) {  
        Integer i=1;  
        doSomething(i);  
        doSomething(2.0);  
    }  
  
    public static void doSomething(double num) {  
        System.out.println("double : " + num);  
    }  
  
    public static void doSomething(Integer num) {  
        System.out.println("Integer : " + num);  
    }  
}
```

- 1.Integer : 1 double : 2.0
- 2.double : 1.0 double : 2.0
- 3.double : 1.0 Integer: 2
- 4.None of the above

Explanation :

A is the correct answer.

The method resolution includes three passes: 1. Attempt to locate the correct method without any boxing, unboxing, or vararg invocations. 2. Attempt to locate the correct method with boxing, unboxing, and without any vararg invocations. 3. Attempt to locate the correct method with boxing, unboxing, or vararg invocations.

Question - 32

What is the output?

```
public class Test {  
  
    public static void main(String[] args) {
```



```

        Character c1 = '\u0000';
        Character c2 = '\u0000';
        System.out.println("c1 == c2 : " + (c1 == c2));

    }

}

```

- 1.c1 == c2 : true
- 2.c1 == c2 : false
- 3.Compile with error
- 4.None of the above

Explanation :

A is the correct answer.

No Exp

Question - 33

```

public class SuperClass {
    public ArrayList doIt() {
        ArrayList lst = new ArrayList();
        System.out.println("super doIt()");
        return lst;
    }
}

public class SubClass extends SuperClass{

    public List doIt()
    {
        List lst = new ArrayList();
        System.out.println("subclas doIt()");
        return lst;
    }

    public static void main(String... args)
    {
        SuperClass sb = new SubClass();

        sb.doIt();

    }
}

```

What is the output ?

- 1.subclas doIt()
- 2.super doIt()
- 3.Compile with error

4. None of the above

Explanation :

C is the correct answer.

The return type is incompatible with SuperClass.doIt(). subtype return is eligible in jdk 1.5 for overridden method but not super type return. super class method return ArrayList so subclass overridden method should return same or sub type of ArrayList.

Question - 34

```
public class SuperClass {
    public List doIt(){
        List lst = new ArrayList();
        System.out.println("super doIt()");
        return lst;
    }
}

public class SubClass extends SuperClass{

    public ArrayList doIt()
    {
        ArrayList lst = new ArrayList();
        System.out.println("subclas doIt()");
        return lst;
    }

    public static void main(String... args)
    {
        SuperClass sb = new SubClass();

        sb.doIt();
    }
}
```

What is the output ?

1. subclas doIt()
2. super doIt()
3. Compile with error
4. None of the above

Explanation :

A is the correct answer.

subtype return is eligible in jdk 1.5 for overridden method but not super type return. super class method return List so subclass overridden method should return same or sub type of List.

Question - 35

```
class C{
int i;
public static void main (String[] args) {
    int i;                //1
    private int a = 1;     //2
    protected int b = 1;   //3
    public int c = 1;       //4
    System.out.println(a+b+c); //5
}}
```

- 1.compiletime error at lines 1,2,3,4,5
- 2.compiletime error at lines 2,3,4,5
- 3.compiletime error at lines 2,3,4
- 4.prints 3

Explanation :

B is the correct answer.

The access modifiers public, protected and private, can not be applied to variables declared inside methods.

Question - 36

Which variables can an inner class access from the class which encapsulates it.

- 1.All final variables
- 2.All instance variables
- 3.Only final instance variables
- 4.All of the above

Explanation :

D is the correct answer.

Inner classes have access to all variables declared within the encapsulating class.

Question - 37

```
class c1
{
    public void m1()
    {
        System.out.println("m1 method in C1 class");
    }
}
class c2
```

```

{
    public c1 m1()
    {
        return new c1(){
            public void m1()
            {
                System.out.println("m1 mehod in anonymous class");
            }
        };
    }
    public static void main(String a[])
    {
        c1 ob1 =new c2().m1();
        ob1.m1();
    }
}

```

- 1.prints m1 method in C1 class
- 2.prints m1 method in anonumous class
- 3.compile time error
- 4.Runtime error

Explanation :

B is the correct answer.

the anonymous class overrides the m1 method in c1.so according to the dynamic dispatch the m1 method in the anonymous is called

Question - 38

What is the output?

```

import java.util.Iterator;
import java.util.TreeSet;

public class Test {
    public static void main(String... args) {

        TreeSet s1 = new TreeSet();
        s1.add("one");
        s1.add("two");
        s1.add("three");
        s1.add("one");

        Iterator it = s1.iterator();
        while (it.hasNext() ) {
            System.out.print( it.next() + " " );
        }

    }
}

```

- 1.one three two

- 2.Runtime Exception
- 3.one three two one
- 4.one two three

Explanation :

A is the correct answer.

B TreeSet assures no duplicate entries.it will return elements in natural order, which, for Strings means alphabetical.

Question - 39

In the bellow code ,

```
public class Test {  
  
    public static void main(String argv[]){  
  
        ArrayList alist = new ArrayList();  
        alist.add("sa");  
        alist.add("ga");  
        alist.add("aa");  
        alist.add("da");  
        // Sort ArrayList code  
  
    }  
  
}
```

How to sort the ArrayList ?

- 1.Collections.sort(alist);
- 2.Collection.sort(alist);
- 3.alist.sort();
- 4.None of the above

Explanation :

A is the correct answer.

Collections.sort(alist) sort the List ascending order.

Question - 40

In the bellow code ,

```
public class Test {  
  
    public static void main(String argv[]){  
  
        ArrayList alist = new ArrayList();  
        alist.add("sa");  
        alist.add("ga");  
        alist.add("aa");  
  
    }  
  
}
```

```

        alist.add("da");
        // Suffle ArrayList code
    }
}

```

How to Suffle the ArrayList ?

- 1.Collections.shuffle(alist);
- 2.Collection.shuffle(alist);
- 3.alist.shuffle();
- 4.None of the above

Explanation :

A is the correct answer.

Collections.shuffle(alist) , suffle the List.

Question - 41

How can we create thread safe List ?

- 1.Collections.synchronizedList(List list);
- 2.Collection.synchronizedList(List list);
- 3.List.synchronizedList();
- 4.None of the above

Explanation :

A is the correct answer.

Collections.synchronizedList(List list) returns Thread safe List.

Question - 42

What is the output ?

```

public class Test {

    public static void main(String argv[]) throws
    InterruptedException{

        int time = Integer.parseInt("5");
        Queue queue = new LinkedList();
        for (int i = time; i >= 0; i--)
            queue.add(new Integer(i));
        while (!queue.isEmpty()) {
            System.out.println(queue.remove());
            Thread.sleep(1000);
        }
    }
}

```

```
}  
  
}
```

- 1.5 4 3 2 1 0
2. Compile time error, Queue not valid Java Interface
3. Runtime Exception
4. None of the above

Explanation :
A is the correct answer.

public interface Queue extends Collection

Question - 43

Which statement is true ?

1. A priority queue does not permit null elements
2. A priority queue does not permit insertion of non-comparable objects
3. A priority queue orders elements according to an order specified at construction time
4. All of the above

Explanation :
D is the correct answer.

A priority queue does not permit null elements. A priority queue does not permit insertion of non-comparable objects. A priority queue orders elements according to an order specified at construction time

Question - 44

What is the Difference between remove() and poll() method of Queue?

1. remove() differs from poll() only in that it throws an exception if this queue is empty.
2. remove() differs from poll() only in that it returns null if this queue is empty.
3. Both are true
4. None of the above

Explanation :
A is the correct answer.

remove(): Retrieves and removes the head of this queue. This method differs from poll only in that it throws an exception if this queue is empty. poll(): Retrieves and removes the head of this queue, or returns null if this queue is empty.

Question - 45

A collection designed for holding elements prior to processing
 ?

- 1.Queue
- 2.ArrayList
- 3.Map
- 4.Vector

Explanation :

A is the correct answer.

Queue : A collection designed for holding elements prior to processing. Besides basic Collection operations, queues provide additional insertion, extraction, and inspection operations.

Question - 46

What is true about Deque?

- 1.A linear collection that supports element insertion and removal at both ends
- 2.A linear collection that supports element insertion and removal at one end
- 3.The name deque is short for "double ended queue"
- 4.1 and 3

Explanation :

D is the correct answer.

The name deque is short for "double ended queue" .A linear collection that supports element insertion and removal at both ends

Question - 47

What is true about Map ?

- 1.A Map is an object that maps keys to values
- 2.A map cannot contain duplicate keys
- 3.Both are true
- 4.None of the above

Explanation :
C is the correct answer.

A Map is an object that maps keys to values. A map cannot contain duplicate keys:

Question - 48

What is the output of the bellow code, if we run using command line

```
import java.util.*;

public class Freq {
    public static void main(String[] args) {
        Map<String, Integer> m = new HashMap<String, Integer>();

        // Initialize frequency table from command line
        for (String a : args) {
            Integer freq = m.get(a);
            m.put(a, (freq == null) ? 1 : freq + 1);
        }

        System.out.println(m.size() + " distinct words:");
        System.out.println(m);
    }
}
```

command line :
java Freq if it is to be it is up to me to delegate

- 1.8 distinct words: {to=3, delegate=1, be=1, it=2, up=1, if=1, me=1, is=2}
- 2.7 distinct words: {delegate=1, be=1, it=2, up=1, if=1, me=1, is=2}
- 3.6 distinct words: {to=3, delegate=1, be=1, it=2, up=1, is=2}
- 4.None of the above

Explanation :
A is the correct answer.

Trace the code your self

Question - 49

What will be the result of compiling the following code:

```
public class SuperClass {
    public int doIt(String str, Integer... data) throws
    ArrayIndexOutOfBoundsException{
        String signature = "(String, Integer[])";
        System.out.println(str + " " + signature);
        return 1;
    }
}
```

```
}
```

```
public class SubClass extends SuperClass{

    public int doIt(String str, Integer... data) throws Exception
    {
        String signature = "(String, Integer[])";
        System.out.println("Overridden: " + str + " " + signature);
        return 0;
    }

    public static void main(String... args)
    {
        SuperClass sb = new SubClass();
        try{
            sb.doIt("hello", 3);
        }catch(Exception e){

        }

    }

}
```

- 1.Overridden: hello (String, Integer[])
- 2.hello (String, Integer[])
- 3.The code throws an Exception at Runtime.
- 4.Compile with error

Explanation :

D is the correct answer.

Exception Exception is not compatible with throws clause in SuperClass.doIt(String, Integer[]). The same exception or subclass of that exception is allowed.

Question - 50

When the application Thrown NullPointerException ?

- 1.Calling the instance method of a null object
- 2.Accessing or modifying the field of a null object
- 3.Taking the length of null as if it were an array
- 4.All of the above

Explanation :

D is the correct answer.

NullPointerException Thrown when an application attempts to use null in a case where an object is required. These include: Calling the instance method of a null object. Accessing or modifying the field of a null object. Taking the length of null as if it were an array. Accessing or modifying the slots of null as if it were an array.

Throwing null as if it were a Throwable value. Applications should throw instances of this class to indicate other illegal uses of the null object.

Question - 51

What will be the result of compiling the following code:

```
public class Test {  
    public static void main(String... args) {  
        Integer.parseInt("hello");  
    }  
}
```

- 1.The code does not compile.
- 2.0
- 3.The code throws an NullPointerException at Runtime.
- 4.java.lang.NumberFormatException at runtime.

Explanation :

B is the correct answer.

NumberFormatException Thrown to indicate that the application has attempted to convert a string to one of the numeric types, but that the string does not have the appropriate format

Question - 52

```
public class A implements Serializable {  
    transient int a = 7;  
    static int b = 9;  
}  
  
public class B implements Serializable {  
    public static void main(String... args){  
        A a = new A();  
        try {  
            ObjectOutputStream os = new ObjectOutputStream(  
                new FileOutputStream("test.ser"));  
            os.writeObject(a);  
            os.close();  
            System.out.print( ++ a.b + " ");  
  
            ObjectInputStream is = new ObjectInputStream(new  
FileInputStream("test.ser"));  
            A s2 = (A)is.readObject();  
            is.close();  
            System.out.println(s2.a + " " + s2.b);  
        } catch (Exception x)
```

```

        {
            x.printStackTrace();
        }

    }
}

```

What is the output?

1. 9 0 9
2. 9 7 9
3. Runtime Exception
4. Compile with error

Explanation :

A is the correct answer.

static and transient variables are not serialized when an object is serialized.

Question - 53

Which is the correct way of Instantiate BufferedWriter object?

1. `BufferedWriter b1 = new BufferedWriter(new File("file.txt"));`
2. `BufferedWriter b1 = new BufferedWriter(new FileWriter("file.txt"));`
3. Both are true
4. None of the above

Explanation :

B is the correct answer.

Constructor of BufferedWriter is `public BufferedWriter(Writer out) { }` so `BufferedWriter b1 = new BufferedWriter(new FileWriter("file.txt"));` is correct one.

Question - 54

What will be the result of compiling and run the following code:

```

public class Test {

    public static void main(String... args) throws Exception {
        Integer i = 34;
        long l = 341;
        if(i.equals(l)){
            System.out.println(true);
        }else{
            System.out.println(false);
        }
    }

}

```

```
}
```

- 1.true
- 2.false
- 3.Compile error
- 4.None of the above

Explanation :

B is the correct answer.

equals() method for the integer wrappers will only return true if the two primitive types and the two values are equal.

Question - 55

What will be the result of compiling and run the following code:

```
public class Test {  
  
    public static void main(String... args) throws Exception {  
        Integer i = 34;  
        int l = 34;  
        if(i.equals(l)){  
            System.out.println(true);  
        }else{  
            System.out.println(false);  
        }  
    }  
}
```

- 1.true
- 2.false
- 3.Compile error
- 4.None of the above

Explanation :

A is the correct answer.

equals() method for the integer wrappers will only return true if the two primitive types and the two values are equal.

Question - 56

When comparing java.io.BufferedWriter and java.io.FileWriter, which capability exist as a method in only one of two ?

- 1.closing the stream
- 2.flushing the stream

3. writing to the stream
4. writing a line separator to the stream

Explanation :

D is the correct answer.

A `newLine()` method is provided in `BufferedWriter` which is not in `FileWriter`.

Question - 57

What will be the result of compiling and run the following code:

```
public class Test {  
  
    public static void main(String... args) throws Exception {  
        File file = new File("test.txt");  
    }  
}
```

1. create new actual file name as test.txt
2. actual file will be not created.
3. Compile error.
4. None of the above

Explanation :

B is the correct answer.

creating a new instance of the class `File`, you're not yet making an actual file, you're just creating a filename

Question - 58

Modifiers Applied to Inner Classes is _____?

1. private
2. final
3. public
4. All of the above

Explanation :

D is the correct answer.

Modifiers Applied to Inner Classes is final abstract public private protected static strictfp

Question - 59

```

class Outer {
    private String x = "Outer variable";
    void doStuff() {
        String z = "local variable";
        class Inner {
            public void seeOuter() {
                System.out.println("Outer x is " + x);
                System.out.println("Local variable z is " + z);
            }
        }
    }
}

```

What is the output?

1. Compile Fail : local variable z is can't access from within inner class
2. Runtime Exception
3. Compile without error.
4. None of the above

Explanation :

A is the correct answer.

the inner class object cannot use the local variables of the method the inner class is in.
only final can be accessible.

Question - 60

```

class Outer {
    private String x = "Outer variable";
    void doStuff() {
        final String z = "local variable";
        class Inner {
            public void seeOuter() {
                System.out.println("Outer x is " + x);
                System.out.println("Local variable z is " + z);
            }
        }
        Inner mi = new Inner();
        mi.seeOuter();
    }

    public static void main(String... args){
        Outer out = new Outer();
        out.doStuff();
    }
}

```

What is the output?

1. Compile Fail : local variable z is can't access from within inner class
2. Outer x is Outer variable Local variable z is local variable

3.Outer x is Outer variable

4.None of the above

Explanation :

B is the correct answer.

the inner class object cannot use the local variables of the method the inner class is in.
only final can be accessible.